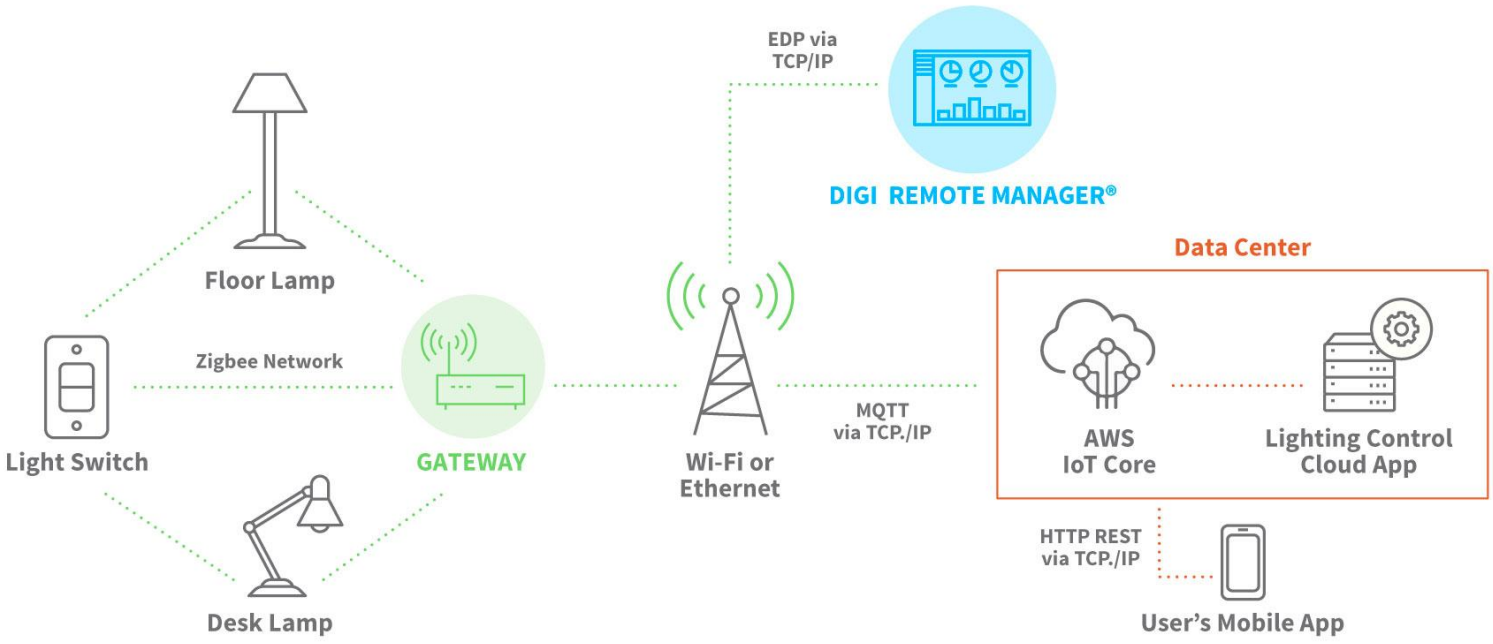


IoT_02 – Communication Protocols in Internet of Things Technical Survey Report

Prepared by: Mohamed Alaa Elshtahy

Field: Internet of Things (IoT)

Type: Self-Study Portfolio

Date: April 2026

Table of Contents

Abstract ..... (i)

1. Introduction ..... (1)

2. Communication Protocols in IoT ..... (2)

    2.1 Definition of Communication Protocols ..... (2)

    2.2 Importance of Protocol Selection ..... (3)

3. MQTT Protocol ..... (4)

    3.1 Overview ..... (4)

    3.2 Publish/Subscribe Model ..... (5)

    3.3 Topics and Quality of Service ..... (6)

    3.4 Advantages and Limitations ..... (6)

4. HTTP Protocol ..... (7)

    4.1 Overview

    4.2 Communication Model

    4.3 Use of HTTP in IoT Systems

    4.4 Advantages and Limitations

5. CoAP Protocol ..... (9)

6. Bluetooth Low Energy (BLE) ..... (10)

7. LoRaWAN Protocol ..... (11)

8. Protocol Comparison ..... (12)

9. Challenges in IoT Communication (13)

10. Conclusion ..... (14)

11. References ..... (15)

Abstract

The Internet of Things (IoT) enables physical devices such as sensors, actuators, and embedded systems to communicate over the Internet and exchange data automatically.

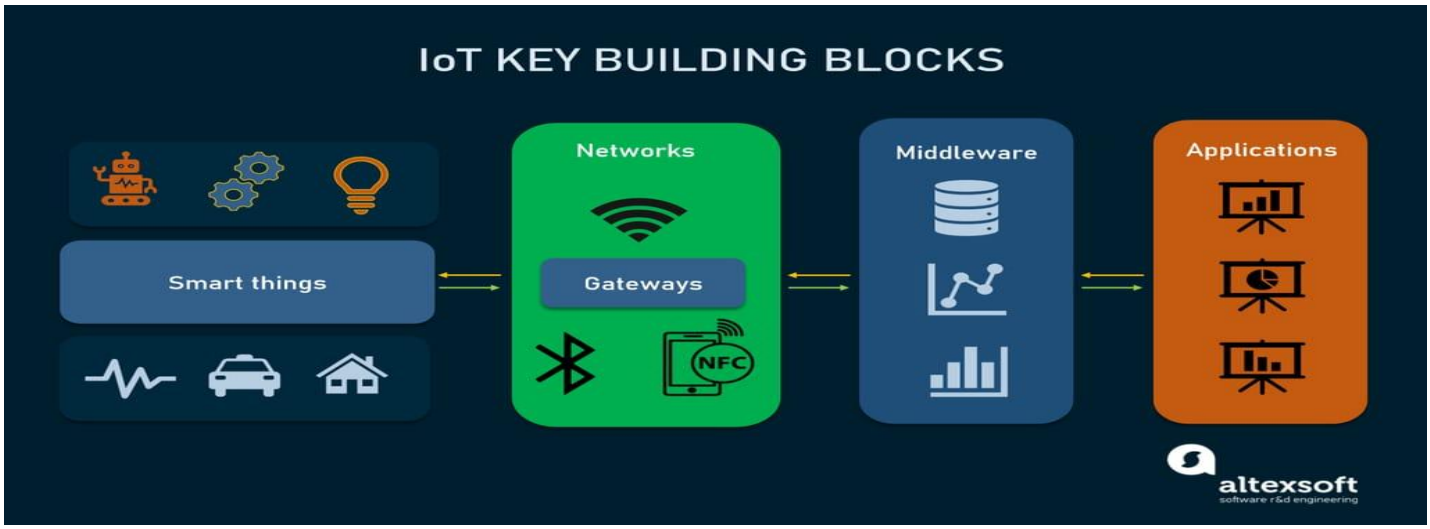
Communication protocols form a fundamental component of IoT systems, as they define how data is transmitted between constrained devices, gateways, and cloud platforms.

Due to limitations in processing power, memory, bandwidth, and energy resources, traditional Internet protocols are often inefficient for IoT applications.

This technical survey report presents an overview of commonly used IoT communication protocols, including MQTT, HTTP, CoAP, Bluetooth Low Energy (BLE), and LoRaWAN.

The report analyzes the communication models, advantages, limitations, and application suitability of each protocol.

The objective of this study is to provide a clear understanding of protocol selection and its impact on IoT system performance and reliability.



1. Introduction

The Internet of Things (IoT) refers to a technological paradigm in which physical devices equipped with sensors and communication interfaces are able to collect data and exchange information over the Internet without continuous human intervention. IoT technologies are widely used in modern applications such as smart homes, industrial automation, healthcare monitoring, and smart agriculture.

A typical IoT system consists of sensing devices, a communication layer, and a cloud or application layer. Among these components, the communication layer plays a crucial role, as it connects resource-constrained devices to gateways and cloud platforms, enabling reliable data transmission.

Due to limited processing power, memory, energy resources, and network bandwidth, traditional Internet communication protocols are often inefficient for IoT environments. As a result, specialized communication protocols have been developed to provide lightweight, scalable, and energy-efficient communication. This technical survey focuses on analyzing widely used IoT communication protocols and comparing their suitability for different application requirements.

2. Communication Protocols in IoT

In any Internet of Things (IoT) system, communication protocols are the element that turns isolated devices into a connected system. No matter how accurate the sensors are or how powerful the cloud platform is, an IoT system becomes useless if devices cannot exchange data efficiently and reliably. For this reason, communication protocols are often considered the backbone of IoT architecture.

Unlike traditional computer networks, IoT communication involves devices with very different capabilities. A system may include tiny battery-powered sensors, microcontrollers with limited memory, wireless gateways, and cloud servers. All of these components must communicate using common rules while operating under strict constraints such as low power availability, unstable networks, and limited bandwidth.

To better understand the role of communication protocols in IoT, consider a simple temperature monitoring system. The sensor measures temperature, the microcontroller processes the reading, and the cloud platform stores and visualizes the data. Without an appropriate communication protocol, the data transmission between these elements would either consume excessive energy, suffer from delays, or fail entirely. Therefore, choosing the right protocol is not a secondary decision but a core design choice.

2.1 Definition of Communication Protocols

A communication protocol can be defined as a set of agreed-upon rules that determine how devices exchange information. These rules answer several critical questions: how data is formatted, when it is sent, how the receiver interprets it, and how errors are detected or corrected. In practice, communication protocols ensure that devices “understand each other” even if they are built by different manufacturers or operate on different hardware platforms.

In IoT systems, communication protocols usually operate at the application layer and are designed with simplicity in mind. Unlike traditional protocols such as HTTP, which assume reliable connections and powerful processors, IoT protocols aim to minimize message size, reduce computation overhead, and allow devices to transmit data only when necessary. This design approach helps extend battery life and improve system scalability.

Communication protocols in IoT can also be categorized based on their communication model. Some protocols rely on direct request-response interactions, where a device actively sends a request and waits for a reply. Others use event-driven models, allowing devices to publish data asynchronously

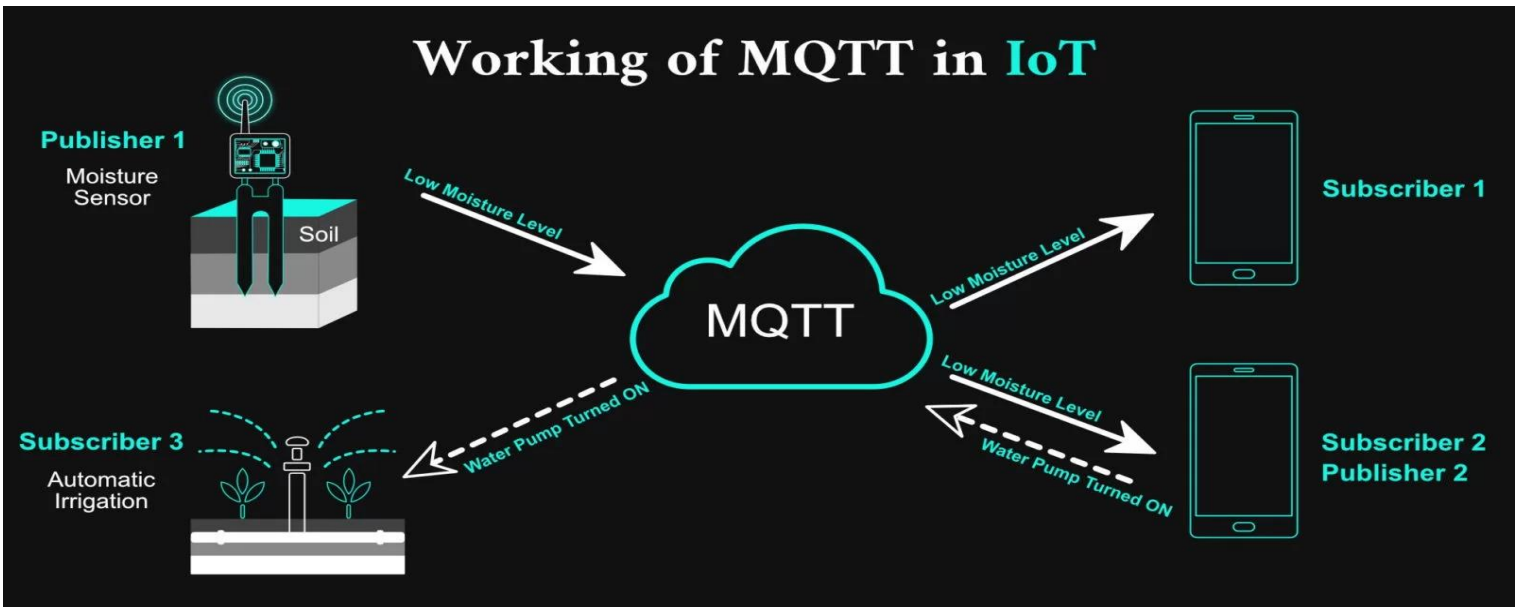
whenever a change occurs. This distinction has a major impact on system latency, power consumption, and real-time performance.

2.2 Importance of Protocol Selection in IoT Systems

Protocol selection is one of the most critical decisions in IoT system design because it directly affects how the system behaves in real-world conditions. A poorly chosen protocol can drain device batteries, overload the network, or introduce unacceptable delays. On the other hand, a well-chosen protocol can significantly improve efficiency, reliability, and scalability.

Different IoT applications require different communication characteristics. For example, a smart home system may prioritize real-time data updates and seamless cloud integration, while a smart agriculture system may require communication over long distances with extremely low power consumption. Wearable devices typically focus on short-range, low-energy communication, whereas industrial IoT systems demand reliable and scalable messaging.

To make an effective protocol choice, engineers must evaluate several factors, including data transmission frequency, message size, acceptable latency, communication range, and energy constraints. There is no single protocol that fits all IoT scenarios. Instead, each protocol represents a trade-off between performance, power consumption, and complexity. Understanding these trade-offs is essential for designing IoT systems that operate efficiently and remain sustainable over long periods.



3. MQTT Protocol

3.1 Overview

Message Queuing Telemetry Transport (MQTT) is a lightweight messaging protocol specifically designed for constrained devices and unreliable or low-bandwidth networks. It was originally developed for remote telemetry and monitoring applications, where network stability could not be guaranteed and power efficiency was critical.

MQTT operates at the application layer and is built on top of the TCP/IP stack. Unlike traditional web protocols such as HTTP, which are request-response oriented and relatively heavy in terms of overhead, MQTT is optimized for minimal packet size and simple control structures. This makes it particularly suitable for IoT devices that have limited processing capabilities, memory constraints, and strict power budgets.

One of the main reasons for the widespread adoption of MQTT in IoT systems is its scalability. MQTT can efficiently support thousands or even millions of devices communicating with cloud platforms simultaneously. For this reason, it is widely used in smart home systems, industrial IoT (IIoT), real-time monitoring, and cloud-based data collection platforms.

3.2 Publish/Subscribe Model

MQTT uses a **publish/subscribe communication model**, which is fundamentally different from the traditional client/server model used in protocols like HTTP. In MQTT, devices do not communicate directly with each other. Instead, all communication is handled through an intermediate entity known as the **broker**.

- **Publisher:** A device or application that sends data.
- **Subscriber:** A device or application that receives data.
- **Broker:** A central server responsible for receiving messages from publishers and delivering them to subscribers.

In this model, publishers send messages to the broker under specific topics, and subscribers receive messages by subscribing to those topics. This design creates a strong decoupling between data producers and consumers. Publishers do not need to know who receives the data, and subscribers do not need to know who sends it. This decoupling simplifies system design and greatly improves scalability.

Compared to HTTP, where a client must continuously send requests to a server, MQTT allows devices to transmit data only when an event occurs. This event-driven communication model significantly reduces network traffic and power consumption, which is critical for battery-powered IoT devices.

3.3 Topics and Quality of Service

Topics

In MQTT, all messages are categorized using **topics**, which are hierarchical strings used to organize data. Topics typically reflect the structure of the IoT system and the type of data being transmitted. For example:

home/livingroom/temperature

factory/machine1/vibration

Subscribers can subscribe to specific topics or use wildcards to receive messages from multiple related topics. This flexible topic structure enables efficient data filtering and routing without unnecessary message transmission.

Quality of Service (QoS)

MQTT provides three **Quality of Service (QoS)** levels that allow developers to balance reliability and resource usage:

- **QoS 0 (At most once):** Messages are delivered without acknowledgment. This level minimizes overhead and power consumption but does not guarantee delivery.
- **QoS 1 (At least once):** Messages are delivered with acknowledgment, ensuring that data is received but allowing possible duplication.

- **QoS 2 (Exactly once):** Messages are delivered exactly once with the highest reliability, at the cost of increased overhead.

This QoS mechanism provides flexibility that is rarely available in traditional IoT communication protocols and allows MQTT to adapt to different application requirements.

3.4 Advantages and Limitations

Advantages of MQTT

- **Low Power Consumption:** Optimized for battery-powered devices.
- **Lightweight Protocol:** Small packet size and minimal control overhead.
- **Scalable Architecture:** Supports large-scale IoT deployments.
- **Event-Driven Communication:** Reduces unnecessary network traffic.
- **Strong Cloud Integration:** Widely supported by cloud IoT platforms.

Limitations of MQTT

- **Broker Dependency:** The broker represents a central point in the architecture. If not properly managed, it may become a single point of failure.
- **TCP Overhead:** Although lightweight, MQTT still relies on TCP, which may be inefficient in extremely constrained or lossy networks.
- **Limited Device-to-Device Communication:** MQTT communication is broker-centric rather than peer-to-peer.

4. HTTP Protocol

4.1 Overview

The Hypertext Transfer Protocol (HTTP) is a widely used application-layer protocol that forms the foundation of communication on the World Wide Web. It was originally designed to enable data exchange between web clients, such as

browsers, and web servers through a simple and standardized mechanism. Due to its global adoption and simplicity, HTTP is often considered when integrating IoT devices with web-based cloud services.

In IoT systems, HTTP is mainly used to connect devices with cloud platforms through RESTful APIs. Many IoT cloud services natively support HTTP, which makes it attractive for initial system prototyping or simple data transmission scenarios. An IoT device can use HTTP to send sensor data to a server, retrieve configurations, or check for firmware updates.

However, although HTTP is convenient and well supported, it was not designed with constrained devices in mind. IoT environments impose strict limitations on memory, processing power, bandwidth, and energy consumption. These limitations expose several inefficiencies when HTTP is used as the primary communication protocol in IoT systems.

4.2 Communication Model

HTTP follows a **client/server request-response model**. In this model, the client initiates communication by sending a request to the server, and the server responds with the requested data. Each request is independent, and the server does not retain the state of previous interactions unless additional mechanisms are implemented.

In the context of IoT, this communication model introduces multiple challenges. IoT devices must continuously send requests even when there is no new data to transmit, which leads to unnecessary network traffic and increased power consumption. In addition, HTTP does not support native server-to-device push mechanisms, making real-time data transmission inefficient.

Compared to MQTT, where devices push data only when an event occurs, HTTP relies heavily on polling. This results in higher latency and increased overhead, especially in large-scale IoT deployments where thousands of devices may communicate simultaneously.

4.3 Use of HTTP in IoT Systems

Despite its limitations, HTTP still plays an important role in many IoT architectures. It is commonly used in scenarios where simplicity, compatibility, and standardization are more important than performance optimization. Typical IoT use cases for HTTP include:

- Device configuration and provisioning
- Firmware over-the-air (OTA) updates
- REST API communication with cloud services
- Non-real-time or periodic data reporting

In modern IoT designs, HTTP is often combined with other protocols. For example, a device may use MQTT for real-time telemetry data while using HTTP for management tasks such as configuration updates or diagnostics. This hybrid approach allows system designers to benefit from HTTP's compatibility without suffering from its performance limitations.

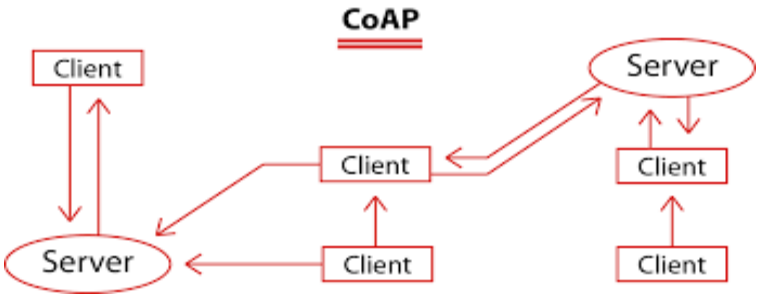
4.4 Advantages and Limitations

Advantages of HTTP in IoT

- Universally supported and well-documented
- Easy integration with cloud platforms and web services
- Simple implementation and debugging

Limitations of HTTP in IoT

- High communication overhead
- Inefficient for real-time communication
- Increased power consumption due to frequent requests
- Not optimized for constrained or battery-powered devices



5. CoAP Protocol

The Constrained Application Protocol (CoAP) is a lightweight application-layer communication protocol designed specifically for resource-constrained IoT devices and low-power networks. CoAP was introduced to overcome the limitations of HTTP when used in environments with restricted processing power, memory, bandwidth, and energy resources.

CoAP follows the same RESTful design principles used in web technologies, such as GET, POST, PUT, and DELETE operations. This makes it conceptually similar to HTTP and easier to integrate with web systems through gateways. However, CoAP significantly reduces protocol overhead, making it more suitable for embedded and battery-powered devices.

Unlike HTTP, CoAP operates over the UDP transport layer instead of TCP. This design choice eliminates the connection establishment overhead and enables faster data transmission with lower power consumption. As a result, CoAP performs well in unreliable or lossy networks, such as wireless sensor networks.

Where CoAP Is Better Than HTTP

- **Lower Power Consumption:** CoAP minimizes communication overhead, which helps extend battery life in constrained devices.
- **Smaller Packet Size:** The protocol uses compact headers, reducing bandwidth usage.
- **Better Performance in Unreliable Networks:** UDP-based communication allows CoAP to function efficiently even when packet loss occurs.

Limitations of CoAP

- **Limited Cloud Support:** CoAP is not natively supported by most cloud platforms compared to HTTP or MQTT.
- **Security Complexity:** Implementing security requires DTLS over UDP, which is more complex than TLS over TCP.

- **Limited Scalability:** CoAP lacks a broker-based architecture, making it less suitable for large-scale IoT deployments when compared to MQTT.

6. Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) is a short-range wireless communication protocol designed for ultra-low power consumption. It was developed to support IoT devices that operate on small batteries and require infrequent data transmission over short distances, such as wearable devices and personal sensors.

BLE is optimized for low-duty-cycle communication, meaning devices remain in sleep mode most of the time and wake up only to transmit or receive small amounts of data. This design significantly extends battery life and makes BLE suitable for devices that must operate for months or years without battery replacement.

Unlike protocols such as MQTT or CoAP, BLE does not rely on Internet connectivity. Communication typically occurs directly between nearby devices, such as a sensor and a smartphone or a local gateway. As a result, BLE is considered a local-area IoT protocol rather than a cloud-centric communication solution.

Key Characteristics of BLE

- **Short Communication Range:** Typically limited to a few meters.
- **Very Low Power Consumption:** Optimized for battery-powered devices.
- **Device-to-Device Communication:** Does not require Internet access.
- **Low Data Rate:** Suitable for small data payloads.

Typical Use Cases

- Wearable devices (fitness trackers, smart watches)
- Medical sensors
- Smart home peripherals
- Personal IoT accessories

Limitations of BLE

- **Limited Range:** Not suitable for wide-area IoT deployments.
- **No Native Cloud Support:** Requires a gateway or smartphone for Internet connectivity.
- **Limited Scalability:** Inefficient for large numbers of devices communicating simultaneously.
- **Low Data Throughput:** Not designed for continuous or high-volume data transfer.

7. LoRaWAN Protocol

LoRaWAN (Long Range Wide Area Network) is a low-power wide-area network (LPWAN) communication protocol designed for IoT systems that require long-range connectivity with extremely low power consumption. It is mainly used in applications where devices send small amounts of data over long distances and operate on batteries for several years.

LoRaWAN uses a **star network topology**, where end devices communicate directly with gateways, which then forward data to a network server and cloud applications. This architecture eliminates the need for complex routing at the device level and significantly reduces power consumption.

Unlike MQTT or HTTP, LoRaWAN is not designed for high data rates or real-time communication. Instead, it prioritizes energy efficiency and coverage. Devices typically transmit data periodically or in response to specific events, making LoRaWAN ideal for remote and outdoor IoT deployments.

Key Characteristics of LoRaWAN

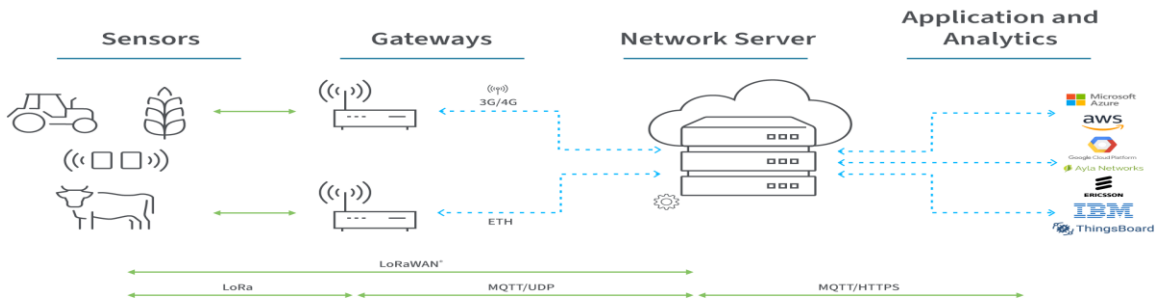
- **Very Long Communication Range:** Can reach several kilometers.
- **Ultra-Low Power Consumption:** Enables multi-year battery life.
- **Low Data Rate:** Optimized for small payloads.
- **Star Network Topology:** Simple and scalable architecture.

Typical Use Cases

- Smart agriculture (soil monitoring, irrigation systems)
- Smart cities (street lighting, parking systems)
- Environmental monitoring
- Utility metering (water, gas, electricity)

Limitations of LoRaWAN

- **Very Low Data Throughput:** Unsuitable for large or frequent data transmissions.
- **High Latency:** Not suitable for real-time applications.
- **Limited Downlink Communication:** Device control capabilities are restricted.
- **Not Designed for Continuous Communication:** Best for periodic or event-based data.



8. Protocol Comparison

Selecting the appropriate communication protocol in an IoT system depends on multiple factors such as communication range, power consumption, data rate, scalability, and real-time requirements. Since no single protocol satisfies all IoT scenarios, understanding the differences between commonly used protocols is essential for effective system design.

Comparison of IoT Communication Protocols

Protocol	Communication Range	Power Consumption	Data Rate	Communication Model	Typical Use Case
MQTT	Medium	Low	Medium	Publish / Subscribe	Cloud-based IoT systems
HTTP	Medium	High	High	Client / Server	APIs, configuration
CoAP	Short	Low	Medium	Request / Response	Constrained sensor networks
BLE	Short	Very Low	Low	Device-to-Device	Wearables, personal devices
LoRaWAN	Very Long	Very Low	Very Low	Star Topology	Smart cities, agriculture

Key Observations

- **MQTT** is the most suitable protocol for large-scale, cloud-centric IoT systems that require real-time data communication and high scalability.
- **HTTP** is best used for configuration, management, and integration with web services rather than continuous data transfer.
- **CoAP** provides a lightweight alternative to HTTP for constrained devices but lacks the scalability and cloud support of MQTT.

- **BLE** is ideal for short-range, battery-powered devices where ultra-low energy consumption is critical.
- **LoRaWAN** excels in scenarios that require long-distance communication with minimal power usage but is unsuitable for real-time or high-data-rate applications.

9. Challenges in IoT Communication

- IoT communication systems face several technical challenges due to the nature of connected devices and the environments in which they operate. These challenges directly impact system performance, reliability, and scalability.

Power Constraints

- Most IoT devices are battery-powered and expected to operate for long periods without maintenance. Frequent data transmission, inefficient protocols, or high communication overhead can significantly reduce battery life. Therefore, minimizing power consumption is one of the most critical challenges in IoT communication.

Network Reliability

- IoT devices are often deployed in remote or harsh environments where network connectivity is unstable or unreliable. Packet loss, interference, and limited bandwidth can affect data transmission reliability and system performance.

Scalability

- Large-scale IoT deployments may involve thousands or millions of devices communicating simultaneously. Managing this volume of communication without network congestion or performance degradation remains a major challenge, especially when using centralized architectures.

Latency

- Some IoT applications, such as industrial automation or real-time monitoring systems, require low latency communication. Ensuring timely data delivery while maintaining low power consumption and reliability is a complex challenge.

Security

- IoT communication systems are vulnerable to various security threats, including eavesdropping, data tampering, and unauthorized access. Implementing strong security mechanisms is difficult due to the limited computational resources of many IoT devices.

10. Conclusion

- Communication protocols are a fundamental component of Internet of Things (IoT) system design. Due to the diverse nature of IoT applications and device constraints, no single communication protocol can satisfy all system requirements. Each protocol represents a specific trade-off between power consumption, communication range, data rate, scalability, and latency.

- This technical survey examined several widely used IoT communication protocols, including MQTT, HTTP, CoAP, Bluetooth Low Energy (BLE), and LoRaWAN. MQTT was identified as the most suitable protocol for cloud-centric and large-scale IoT systems due to its lightweight design and scalable publish/subscribe architecture. HTTP remains useful for device configuration and cloud integration, while CoAP offers a lightweight alternative for constrained environments. BLE is ideal for short-range, ultra-low power applications, and LoRaWAN enables long-range communication with minimal energy consumption.
- Understanding the characteristics, advantages, and limitations of each protocol is essential for selecting the appropriate communication strategy in real-world IoT systems. Proper protocol selection directly impacts system efficiency, reliability, scalability, and long-term sustainability.

---

11. References

1. OASIS. *MQTT Version 3.1.1 Specification*.
2. Fielding, R., et al. *Hypertext Transfer Protocol (HTTP/1.1)*, RFC 2616.
3. Shelby, Z., et al. *Constrained Application Protocol (CoAP)*, RFC 7252.
4. Bluetooth SIG. *Bluetooth Low Energy Specification*.
5. LoRa Alliance. *LoRaWAN Specification*.
6. Tanenbaum, A. S., & Wetherall, D. J. *Computer Networks*, Pearson Education.